

Labo 03 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Trois comportements, une règle

Réponse. La règle : **un conteneur vit exactement le temps de son PID 1**. `alpine` a pour commande par défaut `/bin/sh` ; sans entrée standard, le shell lit une fin de fichier et se termine aussitôt → le conteneur sort. `nginx` reste au premier plan et ne rend jamais la main → le conteneur vit, et bloque votre terminal parce que vous n'avez pas dit `-d`. `-it alpine sh` donne au shell une entrée ouverte et un terminal → il attend vos commandes, donc le conteneur vit tant que vous ne tapez pas `exit`.

Pourquoi. Le moteur ne fait que lancer un processus dans des namespaces et attendre sa fin. Il n'y a pas de notion de « service » : ce qui tient le conteneur en vie, c'est un processus qui ne se termine pas.

Nuance. `podman run nginx` sans `-d` ne signifie pas que `nginx` tourne « différemment » : il est identique, seul votre terminal est attaché à ses sorties. `Ctrl+C` envoie alors `SIGINT` au PID 1 et l'arrête.

Exemple.

```
podman run alpine;                podman ps -a -l --format '{{.Status}}'    # Exited (0)
podman run -d nginx:alpine;        podman ps -l --format '{{.Status}}'    # Up
podman run -it alpine sh -c 'echo "je vis tant que vous voulez"; exit 7'; echo $?    # 7
```

Question 2 — Le `&` qui tue

Réponse. Le script est le PID 1. `java ... &` lance Java en arrière-plan et rend la main immédiatement ; `echo` s'exécute ; le script arrive à sa fin et se termine avec le code `0`. Le PID 1 mort, le noyau tue tout le reste du namespace, Java compris. Correction : lancer Java au premier plan **et** en dernière ligne, avec `exec` :

```
#!/bin/sh
echo "API démarree"
exec java -jar /app/api.jar
```

Pourquoi. `exec` remplace le shell par Java, qui devient PID 1 : il vit aussi longtemps qu'il veut et reçoit directement `SIGTERM`. Sans `exec` mais sans `&`, le script attendrait Java (le conteneur vivrait) mais resterait PID 1 devant lui — et ne relâierait pas `SIGTERM` (question 3 du labo 04).

Nuance. Le code `0` est trompeur : tout s'est « bien passé » du point de vue du script. C'est un exemple de conteneur qui échoue sans erreur — les *restart policies* `on-failure` ne le relanceraient même pas.

Exemple.

```
podman run --rm -v "$PWD":/s alpine /s/demarrage-casse.sh    # revient tout de suite
podman run -d --name ok -v "$PWD":/s alpine /s/demarrage-correct.sh && podman top ok    # sleep
en PID 1
```

Question 3 — Dix secondes et 137

Réponse. `sleep` est le PID 1, et le noyau Linux fait ignorer au PID 1 tout signal pour lequel il n'a pas installé de gestionnaire. `sleep` n'en installe aucun : `SIGTERM` est ignoré. Podman attend le délai de grâce (10 s), annonce qu'il passe à `SIGKILL` — que personne ne peut ignorer — et le processus meurt tué : code `128 + 9 = 137`. `143` (`128 + 15`) n'apparaît que quand c'est `SIGTERM` qui a effectivement terminé le processus.

Pourquoi. Cette protection du PID 1 existe pour qu'un `kill -TERM 1` maladroit ne fasse pas tomber une machine entière. Dans un conteneur, elle se retourne contre vous.

Nuance. Ce n'est pas propre à `sleep` : tout programme sans gestionnaire de `SIGTERM` se comporte ainsi en PID 1 — y compris un script shell, ou un `java` lancé derrière un shell. L'avertissement affiché par Podman (`resorting to SIGKILL`) est précieux : Docker, lui, tue en silence.

Exemple.

```
podman run --rm alpine sh -c 'kill -TERM 1; echo survecu'      # "survecu" : le PID 1 a ignoré
son propre TERM
podman run -d --name v alpine sleep 300; time podman stop v  # 10 s, code 137
```

Question 4 — Ce que change `--init`

Réponse. `--init` insère `podman-init` (un binaire de quelques Ko, `catatonit`) comme PID 1 ; `sleep` devient son enfant, PID 2. `podman-init` sait deux choses : relayer les signaux à son enfant et récolter les zombies. Au `stop`, il reçoit `SIGTERM` et le transmet à `sleep`, qui — n'étant plus PID 1 — subit l'action par défaut : mourir. Code `143`, immédiatement. `podman exec veille ps` montre `1 podman-init` puis `2 sleep`.

Pourquoi. La protection du noyau ne s'applique qu'au PID 1. En déplaçant votre programme au PID 2, on lui rend un comportement normal face aux signaux.

Nuance. `--init` est un pansement : il ne rend pas votre application capable d'un arrêt propre, il la rend seulement *tuable proprement*. Une API Spring Boot gère `SIGTERM` elle-même ; elle n'a pas besoin de `--init`, elle a besoin de le **recevoir** (forme `exec`). `--init` reste utile pour des images qui lancent plusieurs processus et produisent des zombies.

Exemple.

```
podman run --rm --init alpine ps -o pid,comm      # 1 podman-init, 2 ps
```

Question 5 — `-i` sans `-t`, `-t` sans `-i`

Réponse. `-i` maintient `stdin` ouvert et relié à votre clavier ; `-t` alloue un pseudo-terminal (prompt, écho, gestion des touches). `podman run -t alpine sh` : vous voyez un prompt, mais `stdin` n'est pas relié — vos frappes ne sont transmises nulle part, `ls` ne fait rien, et le conteneur reste planté là jusqu'à ce que vous le tuiez depuis un autre terminal (`podman rm -f -t 0`). `podman run -i alpine sh` : pas de prompt ni d'écho, mais ce que vous tapez est transmis : `ls` s'exécute et affiche son résultat, sans confort.

Pourquoi. Ce sont deux canaux indépendants : `-i` concerne le flux de données, `-t` la présentation. Un shell n'a besoin que de `-i` pour fonctionner ; il a besoin de `-t` pour être

agréable.

Nuance. `-i` seul est la forme des scripts : `echo "SELECT 1" | podman exec -i db psql -U app` fonctionne, alors qu'avec `-t` cela échouerait (`the input device is not a TTY`). C'est un bug classique de CI.

Exemple.

```
echo 'echo "recu: $((6*7))"' | podman run -i --rm alpine sh      # recu: 42 – sans prompt
podman run -it --rm alpine sh                                  # prompt "/ #", Ctrl+D pour
sortir
```

Question 6 — `attach` et `Ctrl+C`

Réponse. `attach` a branché son terminal sur les flux du **PID 1** — l'API elle-même. `Ctrl+C` a envoyé `SIGINT` à ce processus, qui s'est arrêté ; le conteneur est mort avec lui. Les deux bonnes façons : `podman logs -f mon-api` (lit les logs captés par `common`, `Ctrl+C` n'arrête que l'affichage) ou `podman exec -it mon-api sh` (nouveau processus, sans effet sur le PID 1).

Pourquoi. `attach` ne crée rien : il reconnecte votre terminal aux tubes existants du processus principal, signaux compris. C'est exactement ce que vous auriez en lançant le conteneur au premier plan.

Nuance. Il existe une porte de sortie : `Ctrl+P` `Ctrl+Q` détache sans arrêter (si le conteneur a été lancé avec `-it`), et `podman attach --sig-proxy=false` empêche la transmission des signaux. Mais la vraie réponse est de ne pas utiliser `attach` pour lire des logs.

Exemple.

```
podman logs -f --tail 20 mon-api      # Ctrl+C : le conteneur continue
podman attach --sig-proxy=false mon-api # Ctrl+C ne sera pas transmis
```

Question 7 — 137, 143, 127

Réponse. `api` (137) : tué par `SIGKILL` — soit un `stop` dont le délai de grâce a expiré, soit l'OOM killer. Confirmer : `podman inspect --format '{{.State.OOMKilled}}' api`, puis `podman events --since 1h | grep api` pour voir s'il y a eu un `stop`. `worker` (143) : a reçu `SIGTERM` et s'est terminé — un arrêt volontaire (déploiement, `podman stop`) ; confirmer avec `podman events` ou `journalctl`. `batch` (127) : la commande n'a pas été trouvée — l'application n'a jamais démarré (erreur d'image ou de `CMD`). Confirmer : `podman logs batch` (message `executable file not found`) et `podman inspect --format '{{.json .Config.Cmd}}' batch`.

Pourquoi. Au-delà de 128, le code est `128 + numéro du signal`. En dessous, c'est le code choisi par le programme — ou par le shell/runtime quand le programme n'a pas pu être lancé.

Nuance. Un `137` avec `OOMKilled: false` et sans `stop` dans les événements peut venir d'un `kill -9` manuel ou d'un orchestrateur. Et le 143 de `worker` en même temps que le 137 d'`api` suggère un arrêt groupé où `api` n'a pas su s'arrêter proprement : c'est le symptôme d'un shell intercalé (labo 04).

Exemple.

```
podman inspect --format '{{.State.OOMKilled}} fini={{.State.FinishedAt}}' api
podman events --since 1h --filter container=api
```

Question 8 — Des logs dans un fichier

Réponse. `podman logs` ne renvoie que ce que `conmon` a capté sur `stdout` / `stderr` du PID 1. En écrivant dans un fichier, l'application contourne ce canal : rien n'est capté. Monter le dossier sur l'hôte rend le fichier lisible, mais reste une mauvaise réponse : les logs échappent à l'outillage (`podman logs`, `journald`, agents de collecte), chaque conteneur invente son chemin, la rotation n'est pas gérée, et un conteneur supprimé laisse des fichiers orphelins.

Pourquoi. Le modèle des conteneurs traite les logs comme un **flux** : le moteur les capte, l'outillage les route (fichier, journal, Loki, Elastic). Un fichier dans le conteneur est un état local, contraire à la nature jetable du conteneur.

Nuance. Spring Boot logue sur la console par défaut : il suffit de **ne pas** définir `logging.file.name`. Si un format fichier est imposé, la solution est un *sidecar* ou un agent qui lit le flux, pas un montage.

Exemple.

```
podman run -d --name l alpine sh -c 'echo visible; echo invisible > /tmp/app.log; sleep 100'
podman logs l                                # visible
podman exec l cat /tmp/app.log                # invisible – seulement en entrant
```

Question 9 — `stop` / `start` contre `rm` / `run`

Réponse. Après `stop` puis `start` : les données sont **conservées** — la couche d'écriture du conteneur existe toujours, PostgreSQL retrouve ses fichiers. Après `rm` puis un nouveau `run` : les données sont **perdues** — `rm` a détruit la couche d'écriture, et le nouveau conteneur repart de l'image.

Pourquoi. `stop` n'agit que sur le processus ; le conteneur (configuration + couche) reste. `rm` supprime l'objet conteneur, couche comprise.

Nuance. L'image `postgres` déclare un `VOLUME` : les données vont dans un volume anonyme qui survit au `rm` mais n'est plus rattaché à rien — irrécupérable en pratique. Le volume nommé (labo 06) est la seule vraie persistance.

Exemple.

```
podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c 'create table t(x int)'
podman stop db && podman start db && podman exec db psql -U postgres -c '\dt'    # t est là
podman rm -f -t 0 db && podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c '\dt'                                       # plus rien
```

Question 10 — `--restart=always` et le reboot

Réponse. Sous Docker, le daemon relit les politiques au démarrage et relance les conteneurs. Sous Podman, il n'y a pas de daemon : `--restart=always` est appliqué par `conmon` tant que le conteneur *existe dans une session vivante*, mais après un reboot, rien ne tourne pour relire la

politique. La manière Podman : un fichier **Quadlet** (`/etc/containers/systemd/api.container` ou `~/.config/containers/systemd/` en rootless) qui décrit le conteneur, et `systemctl enable --now api` — systemd le démarre au boot et le relance sur échec.

Pourquoi. Podman a choisi de ne pas réinventer un gestionnaire de services : Linux en a un, systemd, avec ses dépendances, ses logs et son démarrage au boot. Une *restart policy* Podman ne couvre que la vie d'une session.

Nuance. En rootless, il faut en plus `loginctl enable-linger <utilisateur>` pour que les services de l'utilisateur démarrent sans session ouverte. Sur un poste WSL, tout cela est rarement nécessaire : les conteneurs de développement n'ont pas à survivre au reboot.

Exemple.

```
# ~/.config/containers/systemd/api.container
[Container]
Image=registry.interne/monapp/api:1.4.2
PublishPort=8080:8080
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start api && systemctl --user status api
```

Question 11 — Les logs de la première tentative

Réponse. Dans `podman logs <conteneur>` : les logs s'**accumulent** sur le même conteneur à chaque redémarrage, la première tentative est en haut. `podman restart` ne les efface pas non plus — mais vous perdez `.State.ExitCode` et `.State.FinishedAt` de la dernière exécution, et surtout le conteneur repart en boucle. Regardez d'abord.

Pourquoi. Un redémarrage automatique relance le **même** conteneur (même ID, même couche d'écriture, même fichier de log), il n'en crée pas un nouveau. `podman events` donne en plus la chronologie exacte (`died`, `restart`).

Nuance. Un `podman rm` (ou `--rm`) supprime tout, logs compris. Et un conteneur qui redémarre en boucle peut produire des logs volumineux : `--tail` et `--since` sont vos amis.

Exemple.

```
podman logs --timestamps instable | head -20          # la première exécution
podman events --since 10m --filter container=instable
```

Question 12 — Du moins au plus intrusif

Réponse. (1) `podman inspect` : lit des métadonnées, aucun effet — configuration, état, OOM, PID hôte. (2) `podman logs` : lit ce que `conmon` a déjà capté — ce que l'application dit d'elle-même. (3) `podman stats` : lit les cgroups — CPU, mémoire, I/O réels, sans toucher au conteneur. (4) `podman top` : exécute un `ps` côté hôte sur les PID du conteneur — quel processus consomme, quels threads. (5) `podman exec` : crée un processus **dans** le conteneur — le plus intrusif, mais le seul qui permet un `jstack` ou un `curl localhost:8080/actuator`.

Pourquoi. Les quatre premiers observent depuis l'extérieur, via le moteur ou le noyau ; seul `exec` modifie l'intérieur (un processus de plus, des ressources consommées dans le cgroup du

conteneur).

Nuance. Avec le PID hôte donné par `inspect`, on peut aller plus loin sans `exec` : `cat /proc/<pid>/status`, `strace -p <pid>` — puisque le conteneur est un processus de votre utilisateur en rootless. Et une image *distroless* n'a pas de shell : `exec` n'y est pas possible (labo 05).

Exemple.

```
podman stats --no-stream api
podman top api pid,pcpu,comm
podman exec api jcmd 1 Thread.print | head -50
```

Question 13 — API et base dans le même conteneur

Réponse. Trois conséquences : (1) **un seul PID 1** : il faut un superviseur (`supervisord`) pour tenir deux processus, et si la base meurt, le conteneur ne le sait pas — ou inversement, l'API meurt et emporte la base ; (2) **cycle de vie couplé** : redéployer l'API impose de redémarrer PostgreSQL, avec ses connexions et son cache ; (3) **ressources et observabilité confondues** : une seule limite mémoire, un seul flux de logs mélangé, impossible de mettre l'API à l'échelle sans dupliquer la base.

Pourquoi. Le conteneur est conçu autour d'un processus principal dont la vie est celle du conteneur. Deux processus, c'est deux cycles de vie dans un objet qui n'en a qu'un.

Nuance. Podman a un objet pour « plusieurs conteneurs qui doivent vivre ensemble » : le **pod** (`podman pod create`), qui partage réseau et cycle de vie tout en gardant un conteneur par processus — le même concept que Kubernetes. C'est la réponse correcte au besoin « démarrer simplement ».

Exemple.

```
podman pod create --name stack -p 8080:8080
podman run -d --pod stack --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman run -d --pod stack --name api mon-api:1.0      # joint db sur localhost:5432
```

Question 14 — `--rm` et la production

Réponse. Pour une commande ponctuelle, `--rm` évite d'accumuler des cadavres. Pour un service en production, il détruit à la sortie exactement ce dont on a besoin après un incident : les **logs**, le **code de sortie**, la **couche d'écriture** (fichiers temporaires, *heap dump*), et la possibilité de `podman inspect`. Le conteneur est mort et il n'y a plus rien à examiner. La combinaison avec `--restart` est contradictoire par construction : `--rm` supprime le conteneur à sa sortie, `--restart` veut le relancer à sa sortie — on ne peut pas relancer ce qu'on vient d'effacer. Podman le refuse explicitement.

Pourquoi. Le conteneur `Exited` est l'objet de l'autopsie. Un service qui a planté à 3 h du matin doit pouvoir être inspecté à 9 h.

Nuance. Les orchestrateurs (Kubernetes, Compose) gèrent eux-mêmes la suppression des conteneurs terminés, avec un délai et une rétention des logs. `--rm` reste parfait pour les conteneurs-outils : compilation, migration de base, `psql` interactif.

Exemple.

```
podman run --rm -d --restart=always nginx:alpine
```

```
# Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"
```